

Rockchip Developer Guide Android Boot Video

文件标识: RK-KF-YF-E15

发布版本: V1.0.0

日期: 2024-09-11

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文主要介绍开机视频的制作。

产品版本

芯片名称	内核版本
适配所有芯片	Linux-4.19、Linux-5.10、Linux-6.10

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

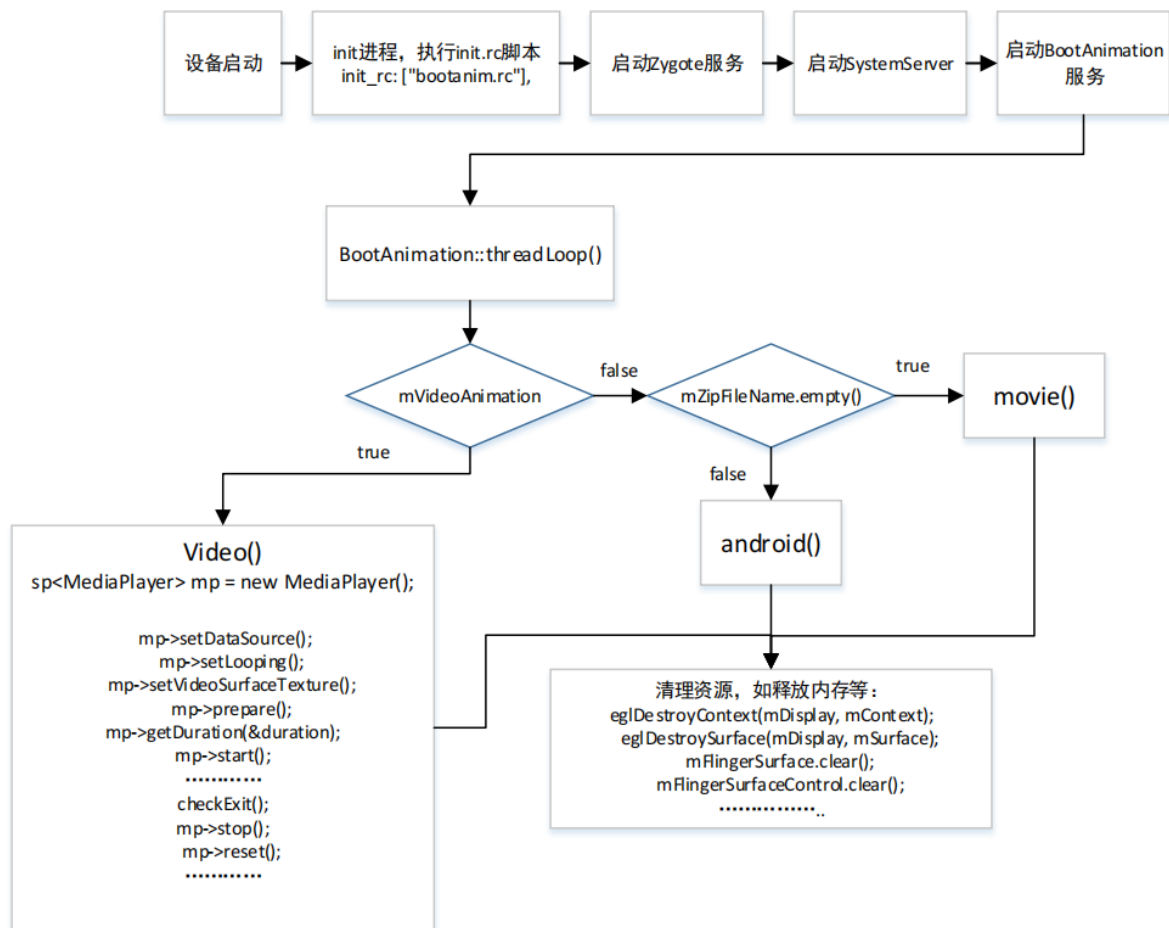
版本号	作者	修改日期	修改说明
V1.0.0	陈祖霖	2024-9-11	初始版本

Rockchip Developer Guide Android Boot Video

1. 开机视频原理
2. 使用方法
 - 2.1 开启和关闭
 - 2.2 开机视频存放位置
 - 2.3 编译
 - 2.4 素材要求
 - 2.5 参数控制说明
3. FAQ
 - 3.1 开机动画显示不全
 - 3.2 开机动画卡顿

1. 开机视频原理

开机动画是 Android 系统启动的一部分，由 bootanimation 负责的，代码路径：
frameworks/base/cmds/bootanimation/BootAnimation.cpp。rk 平台在原生代码的基础上，加入对 boot video 的控制，实现开机视频功能。流程图如下



android() - 系统默认的开机动画

movie() - 用户自定义的开机动画

video() - 用户自定义的开机视频

在 findBootAnimationFile 中会去获取开机的视频文件 bootanimation.ts。视频播放则调用 Android 提供的原生 MediaPlayer 类去实现，视频播放调用 video 函数。在播放动画过程中会调用 checkExit 检查是否要退出动画到 Launcher (即主屏幕界面)。

2. 使用方法

目前 Android14 SDK 已经集成开机视频功能，客户可以直接使用。可以通过阅读 device/rockchip/common/bootvideo/ReadMe.txt，了解操作。

2.1 开启和关闭

客户可以通过在 device/rockchip/common/BoardConfig.mk 中添加开机视频宏

开启配置：

```
BOOT_VIDEO_ENABLE ?= true
```

关闭配置：

```
BOOT_VIDEO_ENABLE ?= false
```

device/rockchip/common/device.mk

```
# boot video enable
ifeq ($(strip $(BOOT_VIDEO_ENABLE)),true)
include device/rockchip/common/bootvideo/bootvideo.mk
endif
```

2.2 开机视频存放位置

将准备好的开机视频素材 bootanimation.ts 放置到 device/rockchip/common/bootvideo/ 目录下。

注：默认代码中按文件名称识别 bootanimation.ts，其它格式例如 bootanimation.mp4 可使用 ffmpeg 转换为 ts 文件类型；

```
ffmpeg -i bootanimation.mp4 bootanimation.ts
```

2.3 编译

整编后，将会把开机视频 bootanimation.ts 拷贝到 out 目录下的 product/media/。针对内部代码，单编 android 经常编译不过，需要加补丁；所以建议在调试的时候，参考如下操作：

1. 把 bootanimation.ts push 到设备的 product/media/
2. 在设备的 product/etc/build.prop，添加下面两个属性

```
persist.sys.bootvideo.enable=true
persist.sys.bootvideo.showtime=-2
```

单编 bootanimation 模块，将 libbootanimation.so push 到设备的 system/lib64。

2.4 素材要求

媒体中心本地应用能够正常播放的视频都可以作为开机视频有效素材，但实际上由于开机视频处于开机阶段，此时系统需要启动大量系统服务，处于高负载状态，为保证开机视频效果，需要根据芯片能力选择视频。（建议选择 1080p 及以下分辨率的素材）

2.5 参数控制说明

开机视频可通过属性控制相关参数：

1. 控制开机视频功能使能属性：persist.sys.bootvideo.enable

```
--true： 开启开机视频
--false： 关闭开机视频
```

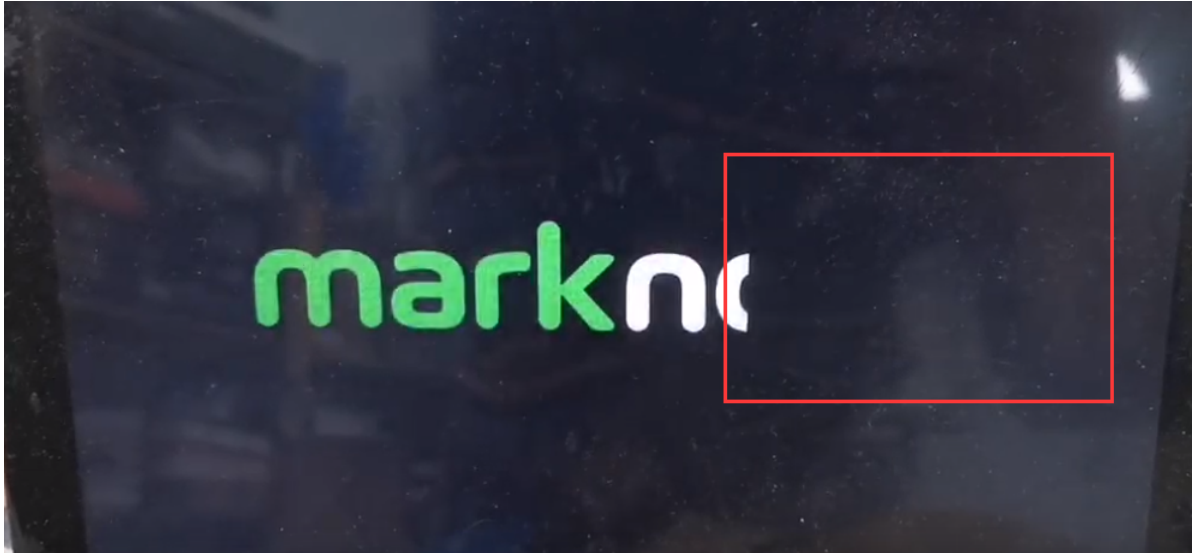
2. 设置开机视频音量属性：persist.sys.bootvideo.vol

参数范围 1-100，表示音量大小，最大音量为 100 刻度。

3. FAQ

3.1 开机动画显示不全

添加开机动画 bootanimation.zip 之后，开机过程，会有一段开机动画右边显示不全的问题，可以检查下 frameworks/base/cmds/bootanimation/BootAnimation.cpp 是否有以下修改；



```
@@ -563,12 +608,18 @@ status_t BootAnimation::readyToRun() {
    mMaxHeight =
    android::base::GetIntProperty("ro.surface_flinger.max_graphics_height", 0);
    ui::Size resolution = displayMode.resolution;
    resolution = limitSurfaceSize(resolution.width, resolution.height);
+   Rect layerStackRect(0, 0, resolution.getWidth(), resolution.getHeight());
+   Rect displayRect(0,0, resolution.getWidth(), resolution.getHeight());
+   SurfaceComposerClient::Transaction t;
+   t.setDisplayProjection(mDisplayToken, ui::ROTATION_0, layerStackRect,
displayRect);
+   t.apply();
    // create the native surface
    sp<SurfaceControl> control = session()-
>createSurface(String8("BootAnimation"),
                resolution.getWidth(), resolution.getHeight(),
PIXEL_FORMAT_RGB_565,
                ISurfaceComposerClient::eOpaque);

-   SurfaceComposerClient::Transaction t;
+   //SurfaceComposerClient::Transaction t;
    if (isValid) {
        // In the case of multi-display, boot animation shows on the specified
displays
        for (const auto id : physicalDisplayIds) {
```

3.2 开机动画卡顿

有些客户开机动画会卡顿，所以可以尝试使用开机视频的方案。但有时会遇到使用开机视频，由于开机阶段的负载很大，导致播放过程会卡顿。这个也和芯片的处理能力有关，例如同一个开机视频，如果使用 rk3588，由于其 cpu 性能很强大，所以播放时不会卡顿；而使用 rk3568 时，就出现卡顿异常了。针对开机视频卡顿，建议图库使用硬解的方式即调用 libhwjpeg 去实现，具体可以参考 usb 摄像头的调用方式。

图库使用硬解方式时，代码实现需要注意的是调整预解码（prepareDecoder）和释放缓存（flushBuffer）的位置，如下：放在构造函数和析构函数里面，预解码（prepareDecoder）只要执行一次就好了。

代码修改路径：frameworks/base/cmds/bootanimation/BootAnimation.cpp。

```
@@ -214,6 +214,13 @@ BootAnimation::BootAnimation(sp<Callbacks> callbacks)
    } else {
        mShuttingDown = true;
    }
+
+    ret = mHWJpegDecoder.prepareDecoder();
+    if (!ret) {
+        ALOGE("failed to prepare JPEG decoder");
+        mHWJpegDecoder.flushBuffer();
+        return NO_ERROR;
+    }
    ALOGD("%sAnimationStartTiming start time: %" PRId64 "ms", mShuttingDown ?
    "Shutdown" : "Boot",
        elapsedRealtime());
}
```

```
BootAnimation::~~BootAnimation() {
+    mHWJpegDecoder.flushBuffer();
    if (mAnimation != nullptr) {
        releaseAnimation(mAnimation);
        mAnimation = nullptr;
@@ -405,6 +413,43 @@ status_t BootAnimation::initTexture(FileMap* map, int*
width, int* height,
    return NO_ERROR;
}
```

创建 initTextureYx 函数，添加对 libhwjpeg 的实现，然后在 playAnimation 中对其引用。initTexture 方法是 BootAnimation 类中的一个成员函数，用于初始化动画中使用的纹理。

```
frameworks/base/cmds/bootanimation/BootAnimation.cpp

++status_t BootAnimation::initTextureYx(FileMap* map, int* width, int* height) {
+    int ret = 0;
+    unsigned int input_len = map->getDataLength();
+    char *srcbuf = (char*)map->getDataPtr();
+
+    if (input_len <= 0) {
+        ALOGE("frame size is invalid !");
+        return -1;
+    }
}
```

```

+     }
+
+     memset(&mHWDecoderFrameOut, 0, sizeof(MpiJpegDecoder::OutputFrame_t));
+     if ((srcbuf[0] == 0xff) && (srcbuf[1] == 0xd8) && (srcbuf[2] == 0xff)) {
+         ret = mHWJpegDecoder.decodePacket(srcbuf, input_len,
+mHWDecoderFrameOut);
+         if (!ret) {
+             ALOGE("mjpeg decodePacket failed!");
+             mHWJpegDecoder.flushBuffer();
+         }
+     } else {
+         ALOGE("mjpeg data error!!");
+         return -1;
+     }
+     // delete map;
+     const int w = mHWDecoderFrameOut.FrameWidth;
+     const int h = mHWDecoderFrameOut.FrameHeight;
+     uint8_t *p = mHWDecoderFrameOut.MemVirAddr;
+
+     glLoadNV12Img(w, h, p);
+     glDrawNV12Img();
+
+     *width = w;
+     *height = h;
+
+     mHWJpegDecoder.deinitOutputFrame(&mHWDecoderFrameOut);
+
+     return NO_ERROR;
+ }
+

```